

BACK-END DEVELOPMENT

WEEK 5 – React + Axios Integration



CADT
IDT

After Finishing This Lecture:

- What is Axios
- Axios basics: **GET/POST** from client
- **useEffect, useState** for loading data
- React component
- Basic form submission
- Displaying and deleting data
- Authorization in Request header

What is **Axios**



Introduction to **Axios**

Axios is a promise-based HTTP client for JavaScript, used to make requests to external resources (like APIs). It works in both the browser and Node.js environments.

Axios is commonly used in frontend applications (like **React**, **Vue**, **Angular**, **etc.**) and backend services (Node.js) to handle API requests.

AXIOS as a Promise

❑ Using **Axios** with `.then()` and `.catch()`

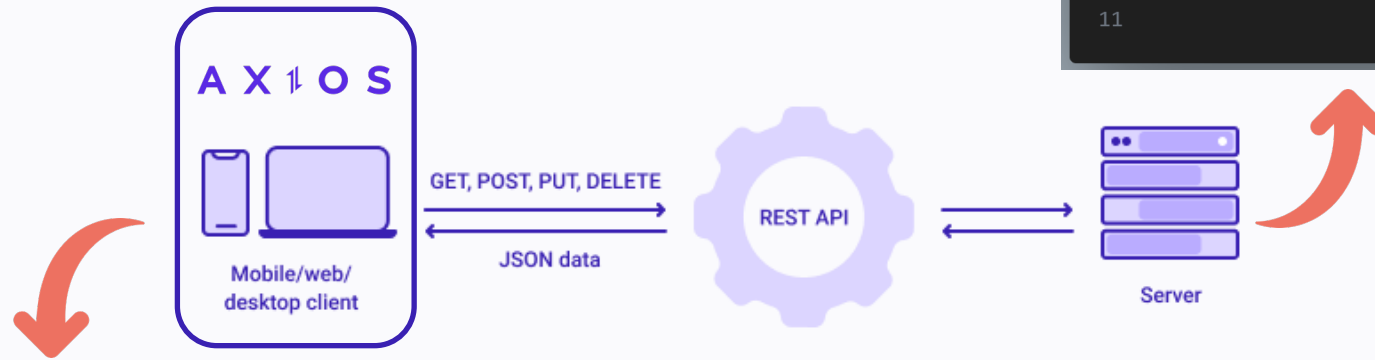
```
1 import axios from 'axios';
2
3 function getUserData() {
4   axios.get('https://api.example.com/user/123')
5     .then(response => {
6       console.log('User Data:', response.data);
7     })
8     .catch(error => {
9       console.error('Error fetching data:', error);
10    });
11 }
12
```



❑ Using **Axios** with `async/await` and `try/catch`

```
1 import axios from 'axios';
2
3 async function getUserData() {
4   try {
5     const res = await axios.get('/user/123');
6     console.log('User Data:', res.data);
7   } catch (error) {
8     console.error('Error fetching data:', error);
9   }
10 }
```

Understanding Axios



```
1
2 app.get('/api/users', (req, res) => {
3   const users = [
4     { id: 1, name: 'Alice Johnson', email: "a@mail.com" },
5     { id: 2, name: 'Bob Martinez', email: "b@mail.com" },
6     { id: 3, name: 'Clara Lee', email: "c@mail.com" }
7   ];
8   res.json(users);
9 }
10 );
11
```

```
1 import axios from "axios";
2
3 axios.get('http://localhost:3000/api/users').then(res => {
4
5   console.log(res.data);
6
7 });
```

Installing Axios

Axios allows developers to easily send **HTTP** requests using methods like **GET**, **POST**, **PUT**, and **DELETE** in NodeJS applications.

❑ Installing Axios

```
$: npm install axios
```

❑ GET Request

```
1 axios.get('https://api.example.com/data')
2   .then(response => {
3     console.log(response.data);
4   })
5   .catch(error => {
6     console.error(error);
7   });
```

Successful response

Error response

❑ POST Request

```
1 axios.post('https://api.example.com/login', {
2   username: 'johndoe',
3   pwd: "1234"
4 })
5 .then(response => {
6   console.log(response.data);
7 })
8 .catch(error => {
9   console.error(error);
10 });
11
```

Request body

Axios usage

Axios Requests with Headers in Node.js



What about passing Headers in a request

HTTP headers are key-value pairs sent with HTTP requests and responses. They carry metadata about the request or the response.

Common uses:

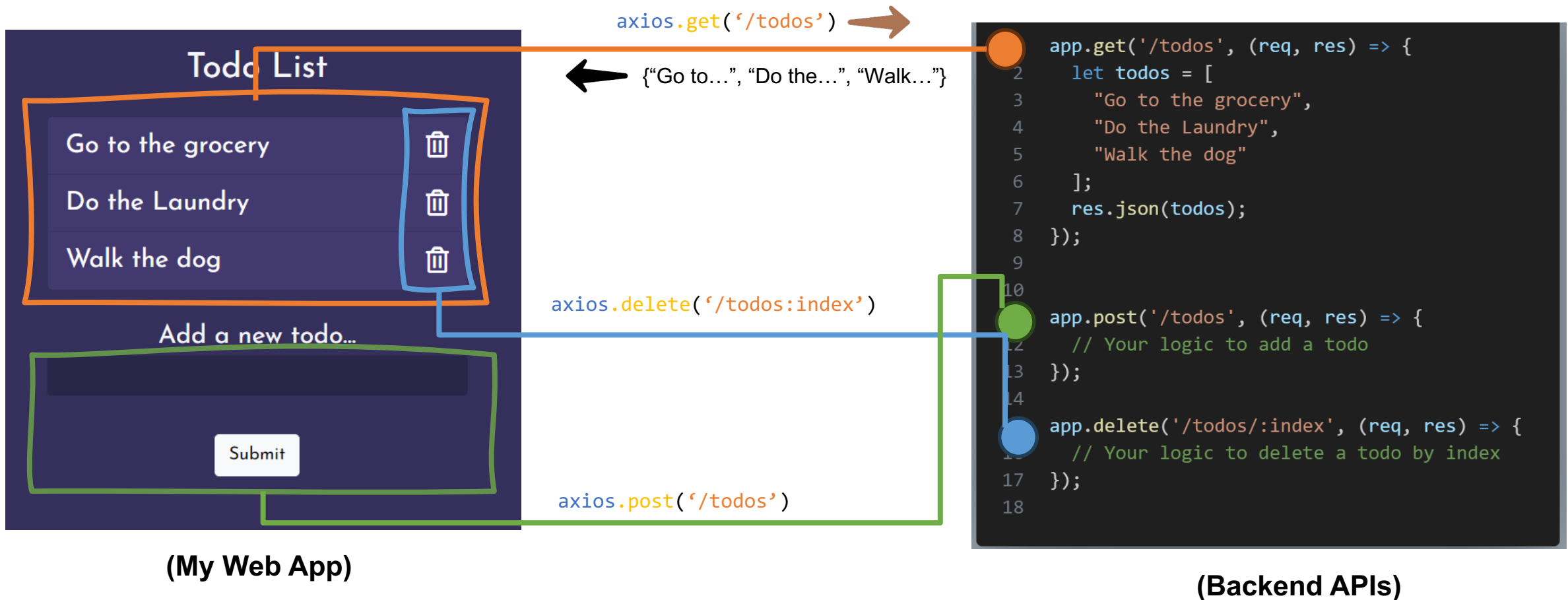
- Sending authentication tokens (e.g., Authorization)
- Specifying data formats (e.g., Content-Type)
- Controlling caching
- Setting custom headers for your API



```
1  const axios = require('axios');
2
3  axios.get('https://api.example.com/users', {
4    headers: {
5      'Authorization': 'Bearer mysecrettoken',
6      'Accept': 'application/json'
7    }
8  })
9  .then(response => {
10    console.log(response.data);
11  })
12  .catch(error => {
13    console.error('Error:', error.message);
14  });
15
```



How to Do an API Integration?



ReactJS

useEffect, useState for loading “Todo List”

```
1  const TodoList = () => {  
2    const [todos, setTodos] = useState([]);  
3    const [loading, setLoading] = useState(true);  
4  
5    useEffect(() => {  
6      // Here we are fetching the todos from the server  
7    }, []);  
8  
9    if (loading) { return <p>Loading todos...</p>; }  
10  
11    return (  
12      <div>  
13        <h2>Todo List</h2>  
14        <ul>  
15          {todos.map((todo, index) => (  
16            <li key={index}>{todo}</li>  
17          ))}  
18        </ul>  
19      </div>  
20    );  
21  };  
22  
23  export default TodoList;
```

```
axios.get('http://localhost:3000/todos')  
  .then(response => {  
    setTodos(response.data);  
    setLoading(false);  
  })  
  .catch(error => {  
    console.error('Error fetching todos:', error);  
    setLoading(false);  
  });
```

Set JSON data from an API
to the state

Loading status

Rendering todo list items

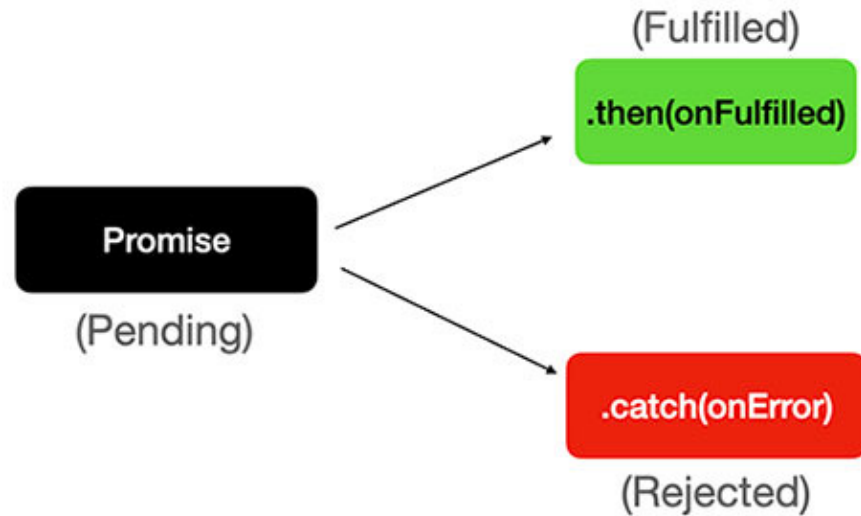
A component in ReactJS

ReactJS

useEffect, useState for loading “Todo List”

Loading Data from APIs

Several actions should be taken:



In response (Todos successfully loaded)

1. Update the UI with the Fetched Data
2. Hide Any Loading Indicators
3. Clear Any Previous Error Messages
4. Saving into cache

In error response:

1. Show an Error Message to the User
2. Show a button to reload/fetch the data again

ReactJS

useEffect, useState for loading “Todo List”

Loading Data from APIs

General Use Cases

Use Case	Description
Product Listings	Fetch product data from an e-commerce backend to display on the store page.
User Profile	Get user details (name, avatar, preferences) to display on a dashboard or profile page.
Blog Posts / News Feed	Load posts or articles from a content API to show on a blog/news site.
Weather Info	Fetch live weather data from an external API (like OpenWeatherMap).
Search Results	Submit a search query to the server and display the matching results.
Comments Section	Load comments related to a specific post or item.
Chat Messages	Fetch recent messages when entering a chat room or conversation thread.
Analytics Dashboard	Display charts and stats using data from analytics APIs.
Task / To-Do List	Load tasks assigned to a user from a task management backend.
Image Gallery	Fetch image metadata (URLs, titles) from a server or cloud service (e.g., Cloudinary).
Many more...	

ReactJS

useState for creating a new “Todo”

```
1  const AddTodo = () => {
2    const [newTodo, setNewTodo] = useState('');
3
4    const handleAddTodo = async (e) => {
5      e.preventDefault();
6      // Here we are sending a POST request to add a new todo
7    };
8
9    return (
10     <div>
11       <h2>Add Todo</h2>
12       <form onSubmit={handleAddTodo}>
13         <input
14           type="text"
15           value={newTodo}
16           onChange={(e) => setNewTodo(e.target.value)}
17           placeholder="Enter a new task"
18         />
19         <button type="submit">Add</button>
20       </form>
21     </div>
22   );
23 };
24
25 export default AddTodo;
```

```
axios.post('http://localhost:3000/todos', { todo: newTodo })
  .then(response => {
    console.log("Todo added successfully:", response.data);
  })
  .catch(error => {
    console.error('Error adding todo:', error);
  });
```

A component in ReactJS

ReactJS

useState for creating a new “Todo”

Creating data in APIs

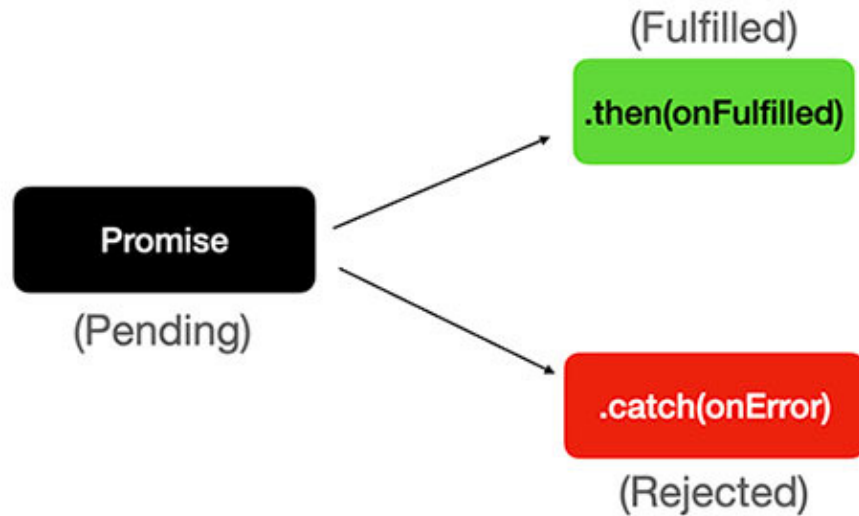
Several actions should be taken:

In response (A Todo successfully added/created)

1. Clear the input field
2. Update the Todo List in the UI
3. Show Feedback (Toast or Message) – (Eg. “A new Todo is added”)

In error response:

1. Show a Clear Error Message to the User
2. Allow the User to Retry



ReactJS

useState for creating a new “Todo”

Creating data in APIs

General Use Cases

Use Case	Description
Sign Up / Register User	Submit new user data to create an account.
Add Product to Store	Admin adds a new product with name, price, and image.
Create New Blog Post	Author submits a new article via a form.
Submit Feedback / Contact Form	User fills out and sends a message to the site.
Add Task to To-Do List	User adds a new task to a personal task manager.
Post a Comment	User comments on a blog post or video.
Upload Photo or File	File input uploads images/documents to the server.
Create Booking/Reservation	User makes a booking for a hotel, event, or service.
Create Invoice	In a billing system, create and store new invoice data.
Many more...	

ReactJS

Deleting a “Todo” by an INDEX

```
1  const DeleteTodoList = () => {
2    const [todos, setTodos] = useState([]);
3    useEffect(() => {
4      // Fetch todos from the server
5    }, []);
6
7    const deleteTodo = (index) => {
8      // Here we are sending a DELETE request to delete a todo
9    };
10
11    return (
12      <div>
13        <h2>Todo List (with Delete)</h2>
14        <ul>
15          {todos.map((todo, index) => (
16            <li key={index}>
17              {todo}{" "}
18              <button onClick={() => deleteTodo(index)}>🗑️</button>
19            </li>
20          ))}
21        </ul>
22      </div>
23    );
24  };
25  export default DeleteTodoList;
```

```
axios.delete(`http://localhost:3000/todos/${index}`)
  .then(() => {
    setTodos(todos.filter((_, i) => i !== index));
    console.log('Todo deleted successfully!');
  })
  .catch(error => {
    console.error('Error deleting todo:', error);
  });
```

Remove the deleted
todo

A component in ReactJS

ReactJS

Deleting a “Todo” by an INDEX

Deleting data in APIs

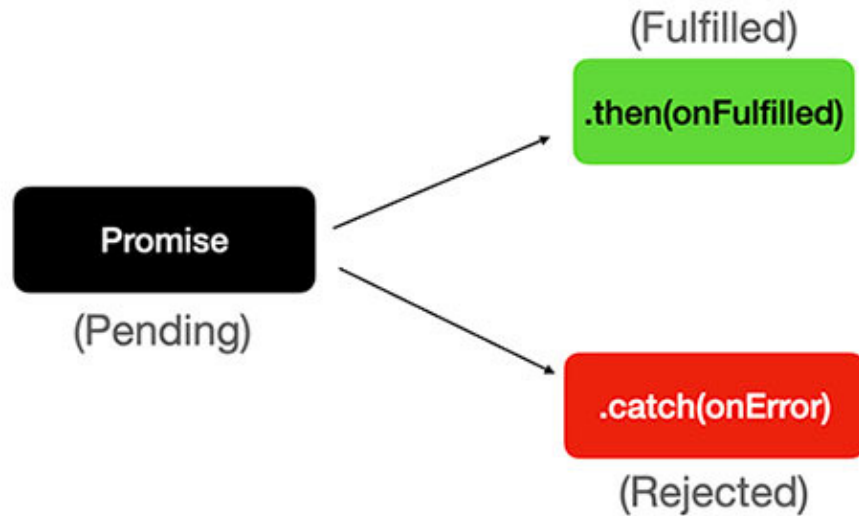
Several actions should be taken:

In response (A Todo successfully removed/deleted)

1. Update the UI Immediately (Re-fetch the List, remove the deleted one)
2. Show a Success Message
3. Reset Any Loading States

In error response:

1. Show a Clear Error Message to the User



ReactJS

Deleting a “Todo” by an INDEX

Deleting data in APIs

General Use Cases

Use Case	Description
Delete User Account	Remove a user and their data from the system.
Remove Product	Admin deletes a product from inventory.
Delete a Comment	User deletes their comment or admin moderates content.
Remove Task	User deletes a to-do item.
Cancel Booking	User cancels an upcoming reservation.
Clear Notifications	Delete read or dismissed notifications.
Remove Item from Cart	Delete an item from the shopping cart.
Delete Uploaded File	Remove an image or document from the server.
Many more...	

Authentication

Basic understanding authentication check
in between **ReactJS** and **ExpressJS**



How to pass an Authentication Bearer token from a React frontend to an **Express.js backend**, using **Axios** and **Authorization** headers.



- ✓ **Frontend (ReactJS):** Sends token using Axios.
- ✓ **Backend (ExpressJS):** Validates the token using middleware.

Authentication

Basic understanding authentication check
in between **ReactJS** and **ExpressJS**



Let's implement it...;

A component in ReactJS

```
1
2 function Dashboard() {
3   useEffect(() => {
4     const token = localStorage.getItem('token'); // or from context/cookies
5
6     axios.get('http://localhost:5000/api/protected', {
7       headers: {
8         Authorization: `Bearer ${token}`
9       }
10    })
11    .then(res => {
12      // Handle the response
13    })
14    .catch(err => {
15      // Handle the error
16    });
17  }, []);
18
19  return <h1>Dashboard</h1>;
20 }
21
22 export default Dashboard;
```



What is Bearer Token Authorization?

Bearer Token Authorization is a way for a client (like a frontend app) to prove its identity to a server by including a special token in the HTTP request headers. You can find following format of it.

Authorization: Bearer <token>

✓ Token typically comes from *localStorage*, *sessionStorage*, *cookie*, or *React Context* after login.

Authentication

Basic understanding authentication check
in between **ReactJS** and **ExpressJS**



Let's implement it...;

An API middleware in ExpressJS

```
1
2 function authenticateTokenMiddleware(req, res, next) {
3   const authHeader = req.headers['authorization'];
4   const token = authHeader && authHeader.split(' ')[1];
5
6   if (!token) {
7     return res.status(401).json({ message: 'No token provided' });
8   }
9
10  if (token !== "my_secret_token") {
11    return res.status(403).json({ message: 'Invalid token' });
12  }
13
14  next();
15 }
16
17 export default authenticateTokenMiddleware;
```

Extract to get just the token string

1. App-Level Middleware

```
const app = express();
app.use(authenticateTokenMiddleware);

app.get('/api/public', (req, res) => {
  res.send('Even this route is now protected!');
});

app.listen(5000, () => console.log('Server is running'));
```

2. Group Route Level (Router-Level Middleware)

```
const app = express();

app.use('/api/protected', authenticateTokenMiddleware);
app.get('/api/protected', (req, res) => {
  res.json({ message: 'Main protected route' });
});

app.listen(5000, () => console.log('Server is running'));
```

3. Route Level (Per Route)

```
app.get('/api/protected', authenticateTokenMiddleware, (req, res) => {
  res.json({ message: 'This is protected data', user: req.user });
});
```

Reflections

Questions



Which HTTP method is used in Express.js to delete a todo item by its index?

- A) GET
- B) POST
- C) PUT
- D) DELETE

Questions



In React, which hook is commonly used to fetch data on component mount?

- A) useState
- B) useEffect
- C) useContext
- D) useReducer

Questions



What is the purpose of `setTodos(response.data)` in a React component after fetching todo data?

- A) To clear the todo list
- B) To update the UI with fetched todos
- C) To send data to the server
- D) To delete all todos

Questions



When *adding a new todo* using **Axios in React, which status code usually indicates a successful creation?**

- A) 200
- B) 201
- C) 400
- D) 500

Questions



What should be done immediately after a todo is successfully deleted in the React state?

- A) Reload the entire page
- B) Remove the deleted todo from local state
- C) Clear all todos from the list
- D) Disable the delete button permanently

Questions



If an Axios request to add a todo **fails due to no server response, what kind of feedback should the app show?**

- A) "Todo added successfully!"
- B) "No response from server. Please check your connection."
- C) "Invalid todo item."
- D) No feedback is necessary

Questions



What does the Express.js route handler for POST /todos expect to receive in the request body?

- A) An array of todos
- B) A todo string under the key *todo* Eg. `{todo: "my task"}`
- C) An ID to delete
- D) Nothing, it uses query parameters

Questions



In React, what is a good practice after successfully adding a todo?

- A) Keep the input field unchanged
- B) Clear the input field for the next entry
- C) Reload the entire page
- D) Disable the add button permanently

Questions



Why might you disable the submit button while an Axios request is in progress?

- A) To prevent duplicate submissions
- B) To speed up the request
- C) To clear the form automatically
- D) To disable the whole form permanently

Additional learnings

- AXIOS Guideline: <https://axios-http.com/docs/intro>
- How To Use Axios with React :
<https://www.digitalocean.com/community/tutorials/react-axios-react>
- **ReactJS: Todo List**
https://www.youtube.com/watch?v=9wiWzu_tRB0&ab_channel=BroCode
- **ExpressJS: Todo List**
https://www.youtube.com/watch?v=2u8VAAyvFv0&ab_channel=TheNormieProgrammer



WHAT WE HAVE LEARNT



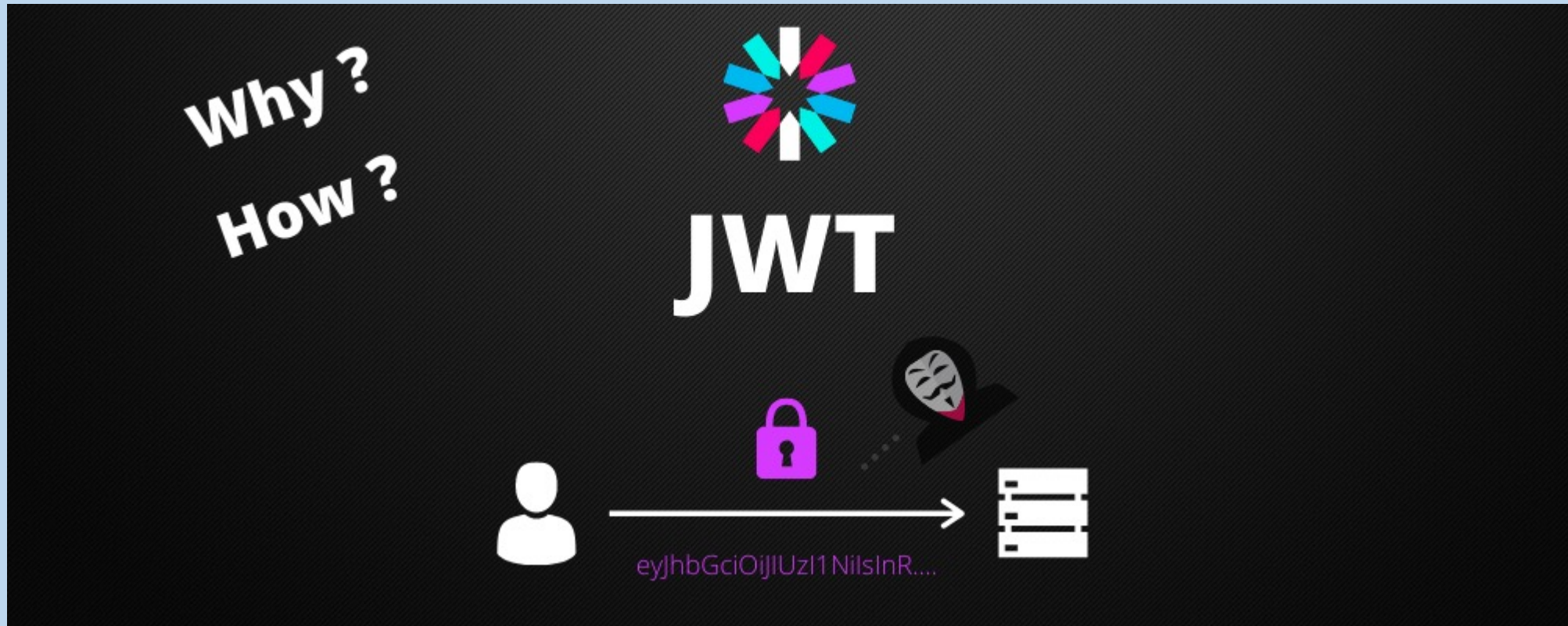
- What is Axios
- Axios basics: **GET/POST** from client
- **useEffect, useState** for loading data
- React component
- Basic form submission
- Displaying and deleting data
- Authorization in Request header



WHAT TO LEARN



JWT: JSON Web Tokens



<https://www.youtube.com/watch?v=7Q17ubqLfaM>